场景接入

本文用一个 KPI 检测实验说明怎样配置、取得数据、调用方法并交付结果，再说明同一机制如何承载其它场景。

1. 一次 KPI 检测实验交付什么

调用方希望回答：**选定的方法对这段服务器 CPU 数据给出了哪些异常分数，这些分数依据什么输入；如果有确认标签，检测质量如何?**

本例选择一段按分钟采样的单服务器 CPU 历史数据。任务要求每分钟用最近 60 点判断当前时点，选用 LibraryTS 中一个符合该任务协议的固定版本方法。GymTS 交付逐时点结果及可追溯的运行记录；评价报告在有明确方案和依据时另行产生。

环节	调用方或 GymTS 做什么	这一环节交出什么
提交计划	调用方选择任务、方法版本、数据来源、变量绑定和运行范围，调用 GymTS.run。	一份 ExperimentSpec；其中引用数据，不装入整段数值。
取得观测	EnvironmentTS 选择回放适配器，定位固定文件版本，按一分钟窗口读取。	ObservationBatch：服务器、CPU、单位、时刻、数值和缺失标记。
整理并运行	Task/Binding 统一变量与单位；积累满 60 点后形成任务输入。方法适配器转换数组布局并调用原生函数。	算法返回的当前时点分数；无需访问文件或数据库连接。
保存并交付	EnvironmentTS 保存实际窗口、方法与适配版本、原始输出及标准任务结果，持续到约定范围结束。	RunResult：结果入口、完成情况、记录入口及产物引用。
按需评价	BenchTS 对齐原始结果与独立标签，按指定匹配规则和指标评分；或在固定案例上重跑方法及基线。	EvaluationReport：指标、基线差异、计分范围与证据引用。

窗口尚未形成时继续收集观测，不产生虚构的检测结果。共同窗口和单位属于任务约定；方法自己的插补、特征处理和阈值计算仍属于被测方法。

一份可读的实验计划

此例假定数据文件已按分钟采样，仅包含服务器 `server-7`；`cpu_pct` 的单位是百分比。适配器、任务和方法名称均为待登记的示例名称，字段格式按接口规范解释。

代码块

```
name: server-cpu-detection-example
scene:
  environment:
```

```yaml
    protocol_version: gymts.environment/1
    adapter: local-kpi-replay
    version: "1.0"
    source:
      kind: dataset
      locator: local://datasets/server-7-cpu
      revision: example-v1
      connection: {}
      credential_ref: null
    mapping:
      time: {field: timestamp, timezone: Asia/Shanghai}
      entity: {field: server_id}
      variables:
        cpu_utilization: {field: cpu_pct, dtype: float64, unit: percent}
    options: {availability: event_time}
  task:
    plugin: point_anomaly_detection
    version: "1.0"
    params:
      variables: [cpu]
      sample_period: 1min
      window_points: 60
      decision_every: 1min
      missing: preserve_mask
      output: point_anomaly_score
  binding:
    variables:
      cpu: {source: cpu_utilization, unit: ratio, transform: divide_by_100}
    operations: {}
candidate:
  adapter: library-window-detector
  version: "1.0"
  artifact: null
  params: {}
execution:
  mode: stream
  start: {mode: reinitialize}
  scope:
    start: "2026-09-01T08:00:00+08:00"
    end: "2026-09-01T10:00:00+08:00"
  learning: frozen
recording:
  inputs: reference
  outputs: full
  artifacts: reference
```

`candidate` 是配置中选方法的位置，对应 EnvironmentTS 内部的一个可调用实例。上例无模型文件，因此 `artifact=null`；算法实现版本由已登记的适配声明确定。

调用方最终拿到的结果可概括为以下结构示例，引用需在实际运行时解析到保存的对象：

代码块

```
run_id: example-run-001
purpose: development
completion: scope_reached
result_ref: local://runs/example-run-001/results
artifact_refs: []
trace_ref: local://runs/example-run-001/events
summary:
  end_reason: configured_scope_reached
```

`result_ref` 中的每项检测结果关联服务器、变量、目标时点、原始输出和输入窗口。调用方可以据此查看某个分数的依据。没有标签时依然能够完成这次运行；正确率等需要标签的指标则不能据此直接给出。独立评分使用 `BenchTS.score`，重新执行测试案例使用 `BenchTS.evaluate`。

2. 更换来源或方法时，具体改哪里

要做的变化	需要改变	可继续复用
同一检测任务，从历史回放换成实时观测	环境适配器和连接来源、指标映射、attach 启动、明确的观测时长；核对采样和可用历史。	检测任务的输入/输出含义、方法调用、结果记录和评价入口。实时源也必须能提供该任务要求的变量与窗口。
同一数据来源，比较另一个检测方法	方法适配名称/版本、参数和需要的模型制品；核对任务协议、窗口及缺失支持。	固定数据版本、任务绑定、运行范围和评分方案。每个方法得到独立的 Run 和结果。
同一来源，从检测改成预测	任务目标、可用历史和预测范围、预测方法、对应评分规则。	已有数据连接、观测获取、运行记录，以及后续结果的关联机制。

实时源若没有预热历史，就先积累本次会话的窗口；不能因为历史实验一开始已有上下文，就假定实时实验也具备。同一接口保证使用方式与含义清楚，不保证不同来源的数据分布或测试难度相同。

3. 五种实践如何映射到同一过程

下面各场景都复用“取得观测 → 构造任务输入 → 运行方法 → 处理输出/请求 → 记录及评价”。变化集中在来源能力、任务含义和评价依据。

ChatTS 合成快照

映射环节	具体接法
数据来源	读取已有固定版本的合成文件，按整段或窗口交付数值，时钟使用采样点。按配方重新生成是另一项生成器适配工作。
运行交互	形态问答、事件定位等任务明确窗口内位置与全局点号的对应。快照读取没有控制动作。
结果与评价	保存原始回答和位置结果；生成属性/标签单独供评分。ChatTS 生成器是数据来源，ChatTS 模型可以是被调用的方法。

阿里 CMS 只读观测

映射环节	具体接法
数据来源	取得获准查询通道后，连接指定数据空间，按实体、指标与分辨率轮询窗口，保存实际响应。只有导出文件时按回放快照接入。
运行交互	attach 建立会话；本版查询只读本次已保存、已开放的历史。窗口未到时等待，finish 结束本次观测。
结果与评价	保存查询条件、窗口和首次获取时间；有独立标签才评价正确率。公开页面不提供“API 已接通”的证明。

原生 Gym 控制环境

映射环节	具体接法
数据来源	包装已有 reset/step：开始时缓存初态，每个动作调用一次 step，随后交付后继状态及奖励。Gym 是接口形态，其后端的物理来源另行说明。
运行交互	方法依据状态选动作，环境产生原生转移；observe 只取缓存，不额外推进。
结果与评价	保存初态、动作、后态、奖励和终止/截断信息。最后状态与奖励先交付再收尾；BenchTS 可评价回合结果。

诊断 Agent 查询

映射环节	具体接法
数据来源	使用已经接入的指标环境，先取得事件及允许的历史窗口。
运行交互	Agent 返回任务级查询请求；GymTS 查得结果后，以同一任务实例的下一轮输入送回，直到形成诊断。
结果与评价	保存每轮请求、当时返回的数据和最终报告，按领域的原因/证据规则评价。Agent 是本次运行的参与者，环境取数仍经 GymTS。

预测与延迟结果

映射环节	具体接法
数据来源	复用 KPI 来源；发出预测时只提供当时可见历史，后续观测继续到达。
运行交互	模型给出带目标时点的预测；预测本身不是环境控制。需要在线学习时，另按任务规则提供已可用反馈。
结果与评价	先固定原预测，后用 records.attach_outcomes 关联目标值；评分不改写原预测。未到达的目标保留为待评价范围。

这组实践中的区别有直接设计含义：快照可以重读，但不会响应新控制；实时系统自行运行，结束观测不等于关闭系统；控制环境的动作会改变后续状态；查询只补充已允许的信息；预测的评价依据可能晚于运行输出。

4. LibraryTS 各类条目

让库里的算法都能参与 GymTS，需要为不同用途提供相应接法。并非每个函数都应包装成一个独立决策者。

LibraryTS 条目	实现路径	GymTS 最终取得什么
检测、预测、诊断等完整方法	声明支持的任务与输入条件，提供薄调用适配；运行时转换为原生参数并包装输出。	与任务目标对齐的结果，或允许的查询/控制请求。
插补、滤波、特征等组件	在方法流水线内组合，记录组件版本；需要单独比较组件时，定义对应变换任务。	完整流水线结果，或变换任务约定的输出。
模型、完整流水线、可调用 Agent	保留原生执行方式，以同一任务边界接收数据、返回结果。Agent 的多轮会话在本次 Run 内持续。	任务结果、交互请求及产生的模型/报告等制品。
数据生成器	先生成固定快照供回放，或由环境适配器在本次开始时调用生成器。若评测生成算法本身，则另设生成任务。	可读观测及生成来源；评分标签与观测分开提供。
指标、匹配与评分函数	由 BenchTS 的 Scorer 包装，接收结果与对应评价依据。	指标、计分范围和逐案例结果。
拟合、训练与搜索过程	单次拟合使用明确训练输入及可选标签；跨轮训练/搜索由上层调用运行与评价入口，显式选择每轮新制品。	训练用运行材料、新模型/策略制品及各轮测试报告。

例如 RL 训练器可以调用 GymTS 收集一段有边界的交互，再通过记录接口取得允许的状态、动作、奖励、后态和结束信息，完成原生更新并保存新策略版本。下一 Run 明确使用该版本；独立评价另按固定方案进行。需要保留并行环境或特殊调度的算法，应在实际接入时适配其训练流程。

进入实现时，一项场景接入交付三件东西即可说清：**实际来源和取数映射、任务与方法的匹配配置、一份能追溯输入与输出的示例运行。** LibraryTS 的真实条目还需逐项完成适配；本页给出的是接入路

径。